

Sistema de Detección de Tumores

Centro Médico - Aplicación de Inteligencia Artificial

Documentación - Sistema de Detección de Tumores

Centro Médico - Aplicación de Inteligencia Artificial

Bienvenido a la documentación del Sistema de Detección de Tumores basado en Inteligencia Artificial.

Índice de Documentación

Arquitectura

Descripción detallada de la arquitectura del sistema, incluyendo:

- Estructura del proyecto
- Tecnologías utilizadas
- Diagrama de componentes
- Flujo de datos

Instalación

Guía completa de instalación:

- Requisitos previos
- Instalación del backend
- Instalación del frontend
- Configuración del entorno

Uso de la Aplicación

Manual de usuario:

- Inicio de sesión
- Entrenamiento de modelos
- Realización de predicciones
- Interpretación de resultados

API Reference

Documentación de la API REST:

- Endpoints disponibles
- Parámetros de entrada
- Respuestas esperadas
- Ejemplos de uso

Modelos de ML

Información sobre los modelos de Machine Learning:

- Random Forest
- XGBoost
- AdaBoost
- Gradient Boosting
- Voting Classifier

Testing

Guía de pruebas:

- Tests unitarios
- Tests de integración
- Tests de UI/UX

Enlaces Rápidos

Recurso	Descripción
Backend API	Documentación Swagger
Frontend	Aplicación web



Resumen del Sistema

SISTEMA DE DETECCIÓN DE TUMORES	
Frontend (React)	Backend (FastAPI)
└─ Login	└─ API REST
└─ Panel de Control	└─ Modelos ML (5)
└─ Formulario	└─ Entrenamiento
└─ Resultados	└─ Predicción



Contacto

Para soporte técnico o consultas, contactar al equipo de desarrollo.

Última actualización: Febrero 2026

Arquitectura del Sistema

Visión General

El Sistema de Detección de Tumores es una aplicación web full-stack que utiliza Machine Learning para clasificar tumores mamarios como benignos o malignos.

Estructura del Proyecto

```
Proyecto_completo/
├── backend/                                     # Servidor FastAPI
│   ├── main.py                                # Punto de entrada de la API
│   ├── requirements.txt                       # Dependencias Python
│   ├── pyrightconfig.json                    # Configuración de tipo
│   └── models/                                # Módulos de ML
│       ├── __init__.py
│       ├── random_forest.py
│       ├── xgboost_model.py
│       ├── adaboost.py
│       ├── gradient_boosting.py
│       └── voting_model.py
│   ├── weights/                              # Pesos de modelos entrenados
│   │   ├── *.joblib                          # Modelos serializados
│   │   └── *_accuracy.json                   # Métricas de accuracy
│   └── venv/                                  # Entorno virtual Python
├── frontend/                                  # Aplicación React
│   ├── public/
│   │   └── index.html
│   ├── src/
│   │   ├── App.js                            # Componente principal
│   │   ├── index.js                          # Punto de entrada
│   │   ├── index.css                         # Estilos globales
│   │   ├── context/
│   │   │   └── AppContext.js                 # Estado global
│   │   └── components/
│   │       ├── Login.js
│   │       ├── ModelSelector.js
│   │       ├── PredictionForm.js
│   │       └── PredictionResult.js
│   └── package.json
└── docs/                                      # Documentación
    ├── index.md
    ├── arquitectura.md
    ├── instalacion.md
    └── uso.md
```

```
|
|   |--- api.md
|   |--- modelos.md
|   |--- testing.md
|
|   tests/
|       |--- backend/
|       |--- frontend/
|
# Pruebas
```

Stack Tecnológico

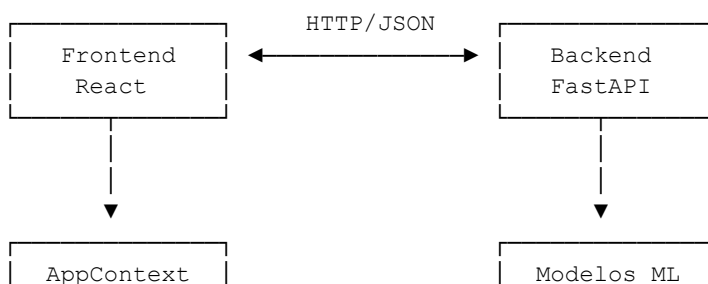
Backend

Tecnología	Versión	Propósito
Python	3.11+	Lenguaje principal
FastAPI	0.109.0	Framework web
Uvicorn	0.27.0	Servidor ASGI
Scikit-learn	1.4.0	Modelos ML
XGBoost	2.0.3	Modelo XGBoost
Pandas	2.2.0	Manipulación de datos
NumPy	1.26.3	Computación numérica
Joblib	1.3.2	Serialización

Frontend

Tecnología	Versión	Propósito
React	18.2.0	Framework UI
Axios	Latest	Cliente HTTP
Context API	-	Estado global
CSS3	-	Estilos

Flujo de Datos



State

sklearn

Flujo de Entrenamiento

1. Usuario selecciona modelo en Frontend
2. Frontend envía POST a /train/{model_name}
3. Backend carga dataset Wisconsin Breast Cancer
4. Entrena el modelo seleccionado
5. Guarda pesos en /weights/
6. Retorna accuracy al Frontend

Flujo de Predicción

7. Usuario ingresa 30 características
 8. Frontend envía POST a /predict
 9. Backend carga modelo entrenado
 10. Realiza predicción
 11. Retorna resultado (Benigno/Maligno) con probabilidades
-

Patrones de Arquitectura

Backend

- RESTful API: Endpoints claros y predecibles
- Modular Design: Cada modelo en su propio módulo
- Dependency Injection: Modelos cargados dinámicamente

Frontend

- Component-Based: UI dividida en componentes reutilizables
 - Context Pattern: Estado global con React Context
 - Responsive Design: Adaptable a diferentes pantallas
-

Seguridad

- CORS: Configurado para localhost:3000
 - Autenticación: Login básico (admin/admin)
 - Validación: Pydantic para validar requests
-

Dataset

Wisconsin Breast Cancer Dataset

- 569 muestras
 - 30 características numéricas
 - 2 clases: Maligno (0) y Benigno (1)
 - Fuente: UCI Machine Learning Repository
-

[← Volver al Índice](#)

Guía de Instalación

Requisitos Previos

Sistema Operativo

- Windows 10/11, macOS, o Linux

Software Necesario

- Python 3.11 o superior
 - Node.js 18.x o superior
 - npm 9.x o superior
 - Git (opcional, para clonar)
-

Instalación del Backend

1. Navegar al directorio del backend

```
cd
C:\Users\Fran\Documents\EOI2025\06_IA\Curso_Python_IA_nov25\Proyecto_completo\backend
```

2. Crear entorno virtual

```
python -m venv venv
```

3. Activar entorno virtual

Windows (PowerShell):

```
.\venv\Scripts\Activate.ps1
```

Windows (CMD):


```
.\venv\Scripts\activate.bat
```

macOS/Linux:

```
source venv/bin/activate
```

4. Instalar dependencias

```
pip install -r requirements.txt
```

5. Verificar instalación

```
python -c "import fastapi; import sklearn; print('OK')"
```

Instalación del Frontend

1. Navegar al directorio del frontend

```
cd  
C:\Users\Fran\Documents\EOI2025\06_IA\Curso_Python_IA_nov25\Proyecto_completo\frontend
```

2. Instalar dependencias

```
npm install
```

3. Verificar instalación

```
npm list react
```

Ejecutar la Aplicación

Iniciar Backend

```
cd backend
.\venv\Scripts\Activate.ps1
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

El backend estará disponible en: <http://localhost:8000>

Iniciar Frontend

En otra terminal:

```
cd frontend
npm start
```

El frontend estará disponible en: <http://localhost:3000>

Verificación

Backend

Abrir en navegador: <http://localhost:8000/docs>

Debe mostrar la documentación Swagger de la API.

Frontend

Abrir en navegador: <http://localhost:3000>

Debe mostrar la pantalla de login.

Solución de Problemas

Error: Puerto 8000 en uso

```
# Encontrar proceso usando el puerto
netstat -ano | findstr :8000

# Terminar proceso
taskkill /F /PID <PID>
```

Error: Módulo no encontrado

```
# Reinstalar dependencias
pip install -r requirements.txt --force-reinstall
```

Error: npm install falla

```
# Limpiar caché
npm cache clean --force
rm -rf node_modules
npm install
```

Error: CORS

Verificar que el frontend esté corriendo en <http://localhost:3000> (no en otro puerto).

Dependencias

requirements.txt (Backend)

```
fastapi==0.109.0
uvicorn==0.27.0
```

```
scikit-learn==1.4.0
xgboost==2.0.3
pandas==2.2.0
numpy==1.26.3
joblib==1.3.2
python-multipart==0.0.6
```

package.json (Frontend)

```
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "axios": "^1.6.0"
  }
}
```

[← Volver al Índice](#)

Manual de Uso

Acceso a la Aplicación

URL de Acceso

- Frontend: `http://localhost:3000`
 - API Docs: `http://localhost:8000/docs`
-

Inicio de Sesión

Credenciales por Defecto

Usuario: `admin`
Contraseña: `admin`

Pasos

12. Abrir `http://localhost:3000`
13. Ingresar usuario: `admin`
14. Ingresar contraseña: `admin`
15. Clic en "Iniciar Sesión"

> Nota: Las credenciales se muestran en la pantalla de login como recordatorio.

Panel de Control

Selección de Modelo

Elegir uno de los 5 modelos disponibles:

Modelo	Descripción
Random Forest	Ensemble de árboles de decisión

XGBoost	Gradient boosting optimizado
AdaBoost	Boosting adaptativo
Gradient Boosting	Boosting por gradiente
Voting Classifier	Combinación de modelos

Entrenamiento

16. Seleccionar modelo del dropdown
17. Clic en "  Entrenar [Modelo]"
18. Esperar a que termine (muestra spinner)
19. Ver mensaje de éxito con accuracy

Estado de Modelos

- ✓ Verde: Modelo entrenado (muestra % accuracy)
 - ○ Gris: Sin entrenar
-



Introducción de Datos

Datos de Ejemplo

Usar los botones rápidos para cargar datos de prueba:

Botón	Descripción
 Caso Benigno	Carga datos de un caso benigno real
 Caso Maligno	Carga datos de un caso maligno real
 Datos Aleatorios	Genera datos aleatorios realistas

Entrada Manual

Completar los 30 campos organizados en 3 secciones:



Valores Medios (Mean)

- Radio medio, Textura media, Perímetro medio
- Área media, Suavidad media, Compacidad media
- Concavidad media, Puntos cóncavos medios
- Simetría media, Dimensión fractal media

Errores Estándar (Error)

- Error de radio, Error de textura, Error de perímetro
- Error de área, Error de suavidad, Error de compacidad
- Error de concavidad, Error de puntos cóncavos
- Error de simetría, Error de dimensión fractal

Valores Peores (Worst)

- Peor radio, Peor textura, Peor perímetro
 - Peor área, Peor suavidad, Peor compacidad
 - Peor concavidad, Peor puntos cóncavos
 - Peor simetría, Peor dimensión fractal
-

Realizar Predicción

Pasos

20. Asegurar que hay un modelo entrenado
21. Cargar o introducir datos
22. Clic en "  Realizar Predicción"
23. Ver resultado

Interpretación del Resultado

Resultado Benigno

Diagnóstico: BENIGNO
Probabilidad Benigno: XX%
Probabilidad Maligno: XX%

El tumor tiene características compatibles con tejido benigno.

Resultado Maligno

Diagnóstico: MALIGNO
Probabilidad Maligno: XX%
Probabilidad Benigno: XX%

El tumor presenta características que sugieren malignidad.

> ⚠ Importante: Este sistema es una herramienta de apoyo. El diagnóstico final debe ser realizado por un profesional médico.

Modo Oscuro/Claro

Cambiar Tema

- Clic en el icono 🌙 / ☀ en la esquina superior derecha
 - El tema se guarda en el navegador
-

Limpiar Formulario

Clic en "🗑 Limpiar" para:

- Vaciar todos los campos
 - Limpiar resultado de predicción
 - Mantener el modelo seleccionado
-

Flujo de Trabajo Típico

1. Login (admin/admin)
- ↓
2. Seleccionar modelo
- ↓
3. Entrenar modelo (si no está entrenado)
- ↓
4. Cargar datos de ejemplo o introducir manualmente
- ↓
5. Realizar predicción
- ↓
6. Interpretar resultado

Atajos de Teclado

Atajo	Acción
Enter	Enviar formulario (realizar predicción)
Tab	Navegar entre campos

Uso en Móvil

La aplicación es responsive y funciona en:

-  Smartphones (portrait y landscape)
-  Tablets
-  Escritorio

[← Volver al Índice](#)

API Reference

Base URL

`http://localhost:8000`

Documentación Interactiva

- Swagger UI: `http://localhost:8000/docs`
 - ReDoc: `http://localhost:8000/redoc`
-

Endpoints

GET /

Información de la API

```
GET / HTTP/1.1
Host: localhost:8000
```

Response 200:

```
{
  "title": "API Detección de Tumores",
  "description": "Sistema de IA para detección de tumores mamarios",
  "version": "1.0.0",
  "endpoints": {
    "GET /models": "Lista de modelos disponibles",
    "POST /train/{model_name}": "Entrenar un modelo",
    "POST /predict": "Realizar predicción",
    "GET /features": "Lista de características requeridas"
  }
}
```

GET /models

Lista de modelos disponibles y su estado

```
GET /models HTTP/1.1
Host: localhost:8000
```

Response 200:

```
[
  {
    "name": "random_forest",
    "display_name": "Random Forest",
    "trained": true,
    "accuracy": 95.61
  },
  {
    "name": "xgboost",
    "display_name": "XGBoost",
    "trained": false,
    "accuracy": null
  }
]
```

POST /train/{model_name}

Entrenar un modelo

Parámetros:

Parámetro	Tipo	Descripción
model_name	path	Nombre del modelo

Valores válidos para model_name:

- random_forest
- xgboost
- adaboost
- gradient_boosting
- voting

```
POST /train/random_forest HTTP/1.1
Host: localhost:8000
```

Response 200:

```
{
  "model": "random_forest",
  "accuracy": 95.61,
  "message": "Modelo Random Forest entrenado y guardado correctamente"
}
```

Response 404:

```
{
  "detail": "Modelo 'invalid_model' no encontrado. Modelos disponibles:
['random_forest', 'xgboost', 'adaboost', 'gradient_boosting', 'voting']"
}
```

POST /predict

Realizar predicción

```
POST /predict HTTP/1.1
Host: localhost:8000
Content-Type: application/json
```

```
{
  "model_name": "random_forest",
  "features": {
    "mean radius": 17.99,
    "mean texture": 10.38,
    "mean perimeter": 122.8,
    "mean area": 1001.0,
    "mean smoothness": 0.1184,
    "mean compactness": 0.2776,
    "mean concavity": 0.3001,
    "mean concave points": 0.1471,
    "mean symmetry": 0.2419,
    "mean fractal dimension": 0.07871,
    "radius error": 1.095,
    "texture error": 0.9053,
    "perimeter error": 8.589,
    "area error": 153.4,
    "smoothness error": 0.006399,
    "compactness error": 0.04904,
    "concavity error": 0.05373,
    "concave points error": 0.01587,
    "symmetry error": 0.03003,
    "fractal dimension error": 0.006193,
    "worst radius": 25.38,
    "worst texture": 17.33,
    "worst perimeter": 184.6,
  }
}
```

```
        "worst area": 2019.0,  
        "worst smoothness": 0.1622,  
        "worst compactness": 0.6656,  
        "worst concavity": 0.7119,  
        "worst concave points": 0.2654,  
        "worst symmetry": 0.4601,  
        "worst fractal dimension": 0.1189  
    }  
}
```

Response 200:

```
{  
  "prediction": "Maligno",  
  "probability_malignant": 99.5,  
  "probability_benign": 0.5  
}
```

Response 400:

```
{  
  "detail": "El modelo 'random_forest' no ha sido entrenado."  
}
```

GET /features

Lista de características requeridas

```
GET /features HTTP/1.1  
Host: localhost:8000
```

Response 200:

```
{  
  "total_features": 30,  
  "feature_names": ["mean radius", "mean texture", ...],  
  "categories": {  
    "mean": ["mean radius", "mean texture", ...],  
    "error": ["radius error", "texture error", ...],  
    "worst": ["worst radius", "worst texture", ...]  
  }  
}
```

GET /sample-data

Datos de ejemplo para pruebas

```
GET /sample-data HTTP/1.1
Host: localhost:8000
```

Response 200:

```
{
  "malignant_example": {
    "mean radius": 17.99,
    "mean texture": 10.38,
    ...
  },
  "benign_example": {
    "mean radius": 12.34,
    "mean texture": 14.56,
    ...
  }
}
```

GET /random-sample

Genera datos aleatorios realistas

```
GET /random-sample HTTP/1.1
Host: localhost:8000
```

Response 200:

```
{
  "random_sample": {
    "mean radius": 14.25,
    "mean texture": 18.72,
    ...
  }
}
```

Ejemplos con cURL

Entrenar modelo

```
curl -X POST "http://localhost:8000/train/random_forest"
```

Realizar predicción

```
curl -X POST "http://localhost:8000/predict" \  
-H "Content-Type: application/json" \  
-d '{"model_name": "random_forest", "features": {...}}'
```

Ejemplos con PowerShell

Entrenar modelo

```
Invoke-RestMethod -Uri "http://localhost:8000/train/random_forest" -Method  
Post
```

Obtener modelos

```
Invoke-RestMethod -Uri "http://localhost:8000/models" -Method Get
```

Códigos de Estado

Código	Descripción
200	Operación exitosa
400	Error de validación o modelo no entrenado
404	Recurso no encontrado
500	Error interno del servidor

[← Volver al Índice](#)

Visión General

El sistema implementa 5 modelos de clasificación, todos basados en el dataset Wisconsin Breast Cancer con 30 características.

Dataset

Wisconsin Breast Cancer Dataset

- Fuente: UCI Machine Learning Repository
- Muestras: 569
- Características: 30 (numéricas)
- Clases: 2 (Maligno=0, Benigno=1)
- Distribución: 357 benignos, 212 malignos

Características

Las 30 características se calculan a partir de imágenes digitalizadas de aspiración con aguja fina (FNA) de masas mamarias:

Categoría	Características
Mean	Radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos cóncavos, simetría, dimensión fractal
Error	Error estándar de las 10 características anteriores
Worst	Peor valor (mayor) de las 10 características base

Random Forest

Descripción

Ensemble de múltiples árboles de decisión que votan para la predicción final.

Hiperparámetros

```
RandomForestClassifier(  
    n_estimators=100,  
    max_depth=10,  
    random_state=42  
)
```

Ventajas

-  Robusto al overfitting
-  Maneja bien datos no lineales
-  No requiere normalización
-  Proporciona importancia de características

Accuracy Típico

~95-97%

XGBoost

Descripción

Implementación optimizada de Gradient Boosting con regularización.

Hiperparámetros

```
XGBClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=10,  
    eval_metric='logloss',  
    random_state=42  
)
```

Ventajas

-  Alto rendimiento
-  Regularización incorporada

-  Manejo de valores faltantes
-  Velocidad de entrenamiento

Accuracy Típico

~95-98%

AdaBoost

Descripción

Boosting adaptativo que ajusta pesos de muestras mal clasificadas.

Hiperparámetros

```
AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(max_depth=10),  
    n_estimators=100,  
    random_state=42  
)
```

Ventajas

-  Simple de implementar
-  Menos propenso al overfitting
-  Buena interpretabilidad
-  Funciona bien con estimadores débiles

Accuracy Típico

~94-96%

Gradient Boosting

Descripción

Construye modelos secuencialmente minimizando errores residuales.

Hiperparámetros

```
GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=10,  
    random_state=42  
)
```

Ventajas

-  Alta precisión
-  Flexible en función de pérdida
-  Bueno para datos tabulares
-  Maneja interacciones complejas

Accuracy Típico

~95-97%

Voting Classifier

Descripción

Combina predicciones de múltiples modelos mediante votación suave.

Composición

```
VotingClassifier(  
    estimators=[  
        ('dt', DecisionTreeClassifier(max_depth=10)),  
        ('lr', LogisticRegression(max_iter=10000)),  
        ('knn', KNeighborsClassifier(n_neighbors=5))  
    ],  
    voting='soft'  
)
```

Modelos Base

- 24. Decision Tree: Ábol de decisión simple
- 25. Logistic Regression: Clasificador lineal probabilístico
- 26. KNN: K-vecinos más cercanos

Ventajas

- ✔ Reduce varianza
- ✔ Más robusto que modelos individuales
- ✔ Aprovecha fortalezas de cada modelo
- ✔ Votación suave para probabilidades

Accuracy Típico

~94-96%

Comparativa de Modelos

Modelo	Accuracy	Velocidad	Interpretabilidad
Random Forest	★★★★★	★★★★	★★★
XGBoost	★★★★★	★★★★★	★★
AdaBoost	★★★★	★★★★	★★★
Gradient Boosting	★★★★★	★★★	★★
Voting	★★★★	★★★	★★★★

Proceso de Entrenamiento

```
1. Cargar dataset (sklearn.datasets.load_breast_cancer)
  ↓
2. Dividir datos (80% train, 20% test, stratified)
  ↓
3. Entrenar modelo con hiperparámetros definidos
  ↓
4. Evaluar en conjunto de test (accuracy)
  ↓
5. Guardar modelo (joblib) y accuracy (JSON)
```

Almacenamiento

Pesos de Modelos

```
backend/weights/  
├── random_forest.joblib  
├── xgboost.joblib  
├── adaboost.joblib  
├── gradient_boosting.joblib  
├── voting.joblib  
└── feature_names.joblib
```

Métricas

```
backend/weights/  
├── random_forest_accuracy.json  
├── xgboost_accuracy.json  
├── adaboost_accuracy.json  
├── gradient_boosting_accuracy.json  
└── voting_accuracy.json
```

Recomendaciones de Uso

Caso de Uso	Modelo Recomendado
Mayor precisión	XGBoost
Equilibrio precision/velocidad	Random Forest
Interpretabilidad	Voting Classifier
Recursos limitados	AdaBoost
Datasets grandes	XGBoost

[← Volver al Índice](#)

Guía de Testing

Visión General

El proyecto incluye pruebas para asegurar la calidad del código, la funcionalidad correcta y una buena experiencia de usuario.

Estructura de Tests

```
tests/
├── backend/
│   ├── test_api.py           # Tests de endpoints API
│   ├── test_models.py       # Tests de modelos ML
│   └── test_integration.py   # Tests de integración
└── frontend/
    ├── test_components.js    # Tests de componentes React
    ├── test_usability.js     # Tests de usabilidad
    └── test_responsive.js    # Tests de responsividad
```

Backend Testing

Ejecutar Tests

```
cd backend
.\venv\Scripts\Activate.ps1
pip install pytest pytest-asyncio httpx
pytest tests/ -v
```

Tests de API

- Verificar endpoints responden correctamente
- Validar esquemas de respuesta
- Probar casos de error

Tests de Modelos

- Verificar entrenamiento exitoso
 - Validar predicciones
 - Comprobar persistencia de pesos
-

Frontend Testing

Ejecutar Tests

```
cd frontend
npm test
```

Tests de Componentes

- Renderizado correcto
- Manejo de estado
- Interacciones de usuario

Tests de Usabilidad

- Flujo de login
- Entrenamiento de modelos
- Proceso de predicción

Tests de Responsividad

- Móvil (320px - 480px)
 - Tablet (768px - 1024px)
 - Desktop (1024px+)
-

Checklist de Testing Manual

Funcionalidad

- ☐ Login con credenciales correctas
- ☐ Login rechaza credenciales incorrectas
- ☐ Logout funciona correctamente
- ☐ Selección de modelo funciona
- ☐ Entrenamiento de cada modelo
- ☐ Muestra accuracy después de entrenar
- ☐ Cargar datos de ejemplo (Benigno)
- ☐ Cargar datos de ejemplo (Maligno)
- ☐ Generar datos aleatorios
- ☐ Realizar predicción
- ☐ Limpiar formulario
- ☐ Cambiar modo claro/oscurο

Usabilidad (UX)

- ☐ Botones de acción claros y visibles
- ☐ Feedback visual durante carga
- ☐ Mensajes de error descriptivos
- ☐ Mensajes de éxito informativos
- ☐ Navegación intuitiva
- ☐ Formulario fácil de completar
- ☐ Resultados fáciles de interpretar

Responsividad

- ☐ Layout correcto en móvil
- ☐ Layout correcto en tablet
- ☐ Layout correcto en desktop
- ☐ Botones accesibles en táctil
- ☐ Texto legible en todas las pantallas
- ☐ Formulario usable en móvil

Accesibilidad

- ☐ Contraste de colores adecuado
- ☐ Labels en inputs
- ☐ Navegación por teclado
- ☐ Mensajes de estado claros

Casos de Prueba Detallados

TC001: Login Exitoso

Precondición: Usuario no autenticado

Pasos:

1. Ingresar usuario: admin
2. Ingresar contraseña: admin
3. Clic en "Iniciar Sesión"

Resultado esperado: Acceso al panel principal

TC002: Login Fallido

Precondición: Usuario no autenticado

Pasos:

1. Ingresar usuario: wrong
2. Ingresar contraseña: wrong
3. Clic en "Iniciar Sesión"

Resultado esperado: Mensaje de error

TC003: Entrenar Random Forest

Precondición: Usuario autenticado

Pasos:

1. Seleccionar "Random Forest"
2. Clic en "Entrenar"
3. Esperar finalización

Resultado esperado: Mensaje de éxito con accuracy

TC004: Predicción con Datos de Ejemplo

Precondición: Modelo entrenado

Pasos:

1. Clic en "Caso Maligno"
2. Clic en "Realizar Predicción"

Resultado esperado: Predicción "Maligno"

TC005: Modo Oscuro

Precondición: Modo claro activo

Pasos:

1. Clic en icono de luna

Resultado esperado: Interfaz cambia a modo oscuro

Métricas de Calidad

Cobertura de Código

- Backend: >80%
- Frontend: >70%

Tiempos de Respuesta

- Login: <1s
- Entrenamiento: <10s
- Predicción: <2s

Usabilidad

- Tiempo para completar tarea: <2min
 - Errores de usuario: <5%
-

Reporte de Bugs

Plantilla

```
**Título:** [Breve descripción]
**Severidad:** Critico / Alto / Medio / Bajo
**Pasos para reproducir:**
1. ...
2. ...
**Resultado actual:** ...
**Resultado esperado:** ...
**Capturas:** [si aplica]
**Entorno:** [navegador, OS]
```

[← Volver al Índice](#)